FINAL DRAFT v2.1 | 16 October 2026

ReSlot (working name)

Disruption-Aware Timetable Rescheduling and Notification Platform

Computer Engineering Department, Rajarshi Shahu College of Engineering (RSCOE), Pune | A.Y. 2026-27

1. Executive summary

Timetables in a college are drawn once and then constantly broken: a lab loses power, a classroom is booked for an event, a faculty member is absent, an extra lab becomes available. Today the fix is made by hand, one phone call at a time, and nobody checks whether the fix creates a new clash in another year's timetable. **ReSlot** turns that into a controlled workflow.

The coordinator uploads the master timetable each time (every upload is kept as a locked snapshot), then reports what changed for a given date (rooms or labs unavailable, extra rooms or labs available, faculty unavailable). ReSlot finds every affected session, proposes a **minimal-change** reschedule that is checked for conflicts across all years and divisions, gets it approved, publishes the updated timetable for that date, and sends each faculty member and student group a personalised WhatsApp message explaining what changed, when and where.

- **Scope:** Computer Engineering department only (SY, TY and Final Year; 8 divisions), single department, single college.
- **Master-first:** the master timetable is the source of truth and is read-only once active (enforced by the database). Every disruption becomes a date-specific override: effective timetable = master + published overrides. Anything a disruption does not touch stays exactly as the master says.
- **Ripple-safe:** a change in one division can affect other divisions and years through shared rooms, faculty and students, so ReSlot solves the whole day for all years in one model and shows the full blast radius before approval.
- **Safety rule:** nothing is published or sent unless an independent validator confirms the change introduces no new room, faculty or student-group clash (the master's own clashes stay visible as a baseline) and a human approves.
- **Foundation already built:** a tested PostgreSQL schema, the department's master timetable transcribed into 312 sessions, and a conflict detector (Section 14).

2. Problem statement and objectives

Manual rescheduling is slow, error-prone and invisible. Rooms and faculty are shared across SY, TY and Final Year, so a local fix often creates a clash elsewhere; the people affected hear late, or not at all.

Objective	Measure of success
Turn a disruption into a conflict-free updated timetable quickly	Plan proposed in seconds; 100% of published timetables pass the independent validator
Disturb as few classes as possible	Fewest sessions moved; prefer a room change (same time) over a time change
Tell the right people exactly what changed	One personalised message per faculty and per affected group; no duplicates; delivery tracked
Handle ripple effects across divisions and years	Every plan shows its full blast radius; sessions outside it are not touched
Keep a trustworthy record	Master versions, original vs new assignment for every override, approvals and audit log

In scope / out of scope

In scope (MVP)	Out of scope (later)
Master timetable import with human verification (PDF, Excel, CSV)	Other departments, first year, multi-college tenancy
Room / lab / faculty availability for a date and time window, incl. temporary extra rooms and labs	Automatic faculty substitution and workload balancing
Conflict detection and automatic minimal-change rescheduling for one date	Generating a whole new semester timetable from scratch
Approval, publishing of the effective timetable, WhatsApp notifications with delivery tracking	Student mobile app (web view and notifications are enough)
Optional AI assistant that turns a typed request into a structured disruption	Attendance, exam scheduling

3. What the master timetable revealed

The department's A.Y. 2026-27 SEM-I timetable PDF (8 pages: SY-A/B/C, TY-A/B/C, Final Year A/B; there is no first-year sheet) was transcribed into structured records and loaded into PostgreSQL.

Item	Value
Divisions	8 (SY A, B, C; TY A, B, C; Final Year A, B)
Scheduled sessions	312 (from 343 timetable cells; 31 duplicates merged into joint classes)
Session mix	160 lectures, 123 labs, 9 tutorials, 20 other (GFM meeting, EIS-II, co-curricular)
Student groups	67 (8 divisions, 43 batches, 16 elective cohorts)
Rooms and labs	17 listed (6 classrooms, KRC, 10 labs); 16 appear in the grid
Faculty codes / subject codes	34 faculty initials (31 teach in the grid) / 32 subject codes
Fixed (not movable) sessions	56 (MDM, EIS-II, COC-II, Minor (BDA), GFM meeting)
Rows needing human review	62 (missing room, missing initials, corrected typos, ambiguous merged cells)
Clashes inside the master itself	22 room clashes, 19 faculty clashes, 0 student-group clashes (as transcribed)

Modelling challenges found in the PDF

Challenge	Example in the PDF	How ReSlot models it
Merged 2-hour lab blocks with parallel batches	SY-A Thu 09:00-11:00: SA1 SEM (RL), SA2 WT (ML), SA3 DBMS (PGL) in one merged cell	One row per batch, each 09:00-11:00; a block is never split
Elective cohorts in the same period	TY: PE-I DMA and PE-I BACKEND run in parallel for the same division	Elective groups with partition keys: same partition = disjoint students, different partition = may share students
Joint classes across divisions	PE-I BACKEND (SSP) at the same time in TY-A, B and C; NLP in both Final Year divisions	One session linked to many groups (23 such sessions)
Co-teaching and combined initials	SC1 SEM taught by SVK/SW; TY-C DMA cell reads NW/JVS	Many-to-many session-faculty link
Aliases and typos	ML lab written MLL, AML and ML; NW for NAW; SPP for SSP; JS for JVS	Alias tables plus a review flag; originals kept
Missing information	25 cells without faculty initials; 26 sessions without a room	Stored as unknown and flagged; never guessed
Slots the solver must not touch	MDM, EIS-II, COC-II, Minor (BDA), GFM meeting	is_fixed flag

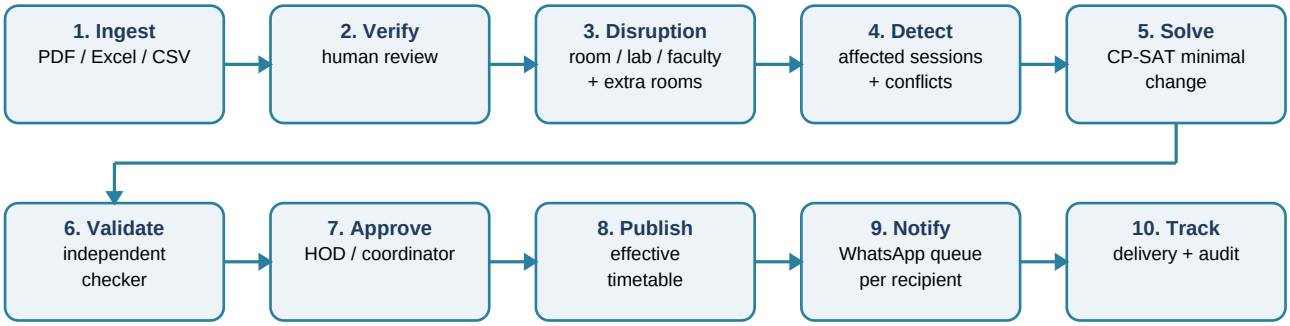
Data-quality and clash findings

- **One classroom, two divisions:** SY-B and TY-A are both assigned A1-202 (clashes on Mon 14:00, Wed 09:00 and 10:00, Sat 09:00 and 10:00); Final Year A and B are both assigned B-216 while having different lectures in the same period.
- **Over-subscribed labs:** Thu 09:00-11:00 the Research Lab (A-212) is used by SY-A, Final A and Final B batches, and the Project Lab (A-211) by SY-B, Final A and Final B batches.
- **Faculty in two places:** e.g. DV on Wed 09:00-11:00 (TY-C lab and Final A lecture); SW on Tue 09:00-11:00 (Final B lab and SY-C lab).
- **Legend and cell mismatches:** Final A lists DBA under SZ but the cells say NK; TY-C EDI-III legend maps PK to Prof. Chandge; IP&PR cells show SSP while the legend says SSS; Ind. Psychology staff is the placeholder XYZ.

Every finding is traceable to a source cell. Because the transcription is mine, a few findings may be reading errors rather than genuine clashes - which is exactly why verification is a first-class step in the product.

4. Solution overview

ReSlot is a pipeline with two human checkpoints: verification of the imported master and approval of every proposed change.



Effective timetable (date) = master timetable (active version, that weekday) + PUBLISHED overrides for that date. The following Monday automatically reverts to the master unless another override exists.

A **disruption** records what happened (A-210 unavailable 09:00-13:15). A **schedule override** records the response (move this session, change its room, or cancel it), always storing the original room and time. Keeping the two separate makes the system auditable and recoverable.

Master-first rules (strict adherence)

- **Read-only master.** Once a version is ACTIVE, the database rejects any edit to its sessions. A correction is a new version that is reviewed and approved.
- **Stored as printed.** The room and faculty initials are kept exactly as written in the PDF (e.g. NW/JVS, SPP, ML). The canonical mapping (NW = NAW) is a labelled interpretation shown in the review screen, never a silent correction.
- **Freeze everything else.** Only sessions hit by a disruption may change; every other session stays exactly as in the master, including its pre-existing clashes.
- **The master's own clashes are the accepted baseline.** ReSlot never edits or hides them; they stay visible as pre-existing. Only a clash created by an override blocks approval.
- **Least departure from the master.** Preference order: same time, another room; then same day, another period; then cancel or flag for a human. Faculty and batch composition are never changed automatically.
- **Unknown stays unknown.** Missing room or initials are flagged for a human decision, never guessed. Each override keeps the original assignment, and the next week returns to the master automatically.

Uploading the master every time

- Each upload becomes a locked snapshot identified by the file's hash. An identical file reuses the existing version; a changed file creates a new one and shows what changed in the master itself.

- Human review decisions (aliases, accepted flags) are saved as rules and re-applied, so only new or changed cells need review (planned for M1; the alias tables exist today).
- Every run records the master version, the inputs and the result, so any output can be reproduced and audited.

5. System architecture

Layer	Choice	Why
Frontend	Next.js + TypeScript	Timetable grids, review screen, approval queue; matches existing skills
Backend	Spring Boot (Java), modular	Business rules, RBAC, approval workflow, scheduler; optimization engine lives behind an interface so it can be extracted later
Optimization	Google OR-Tools CP-SAT (Java)	Exact constraint solving with a weighted objective; avoids a second service in the MVP (a Python service is the fallback)
Database	PostgreSQL 16	Relational integrity, recursive queries for group overlap, partial unique indexes, jsonb audit detail
Notification queue	Postgres outbox table + worker (FOR UPDATE SKIP LOCKED)	No extra infrastructure; idempotent via dedupe keys; Redis can replace it if volume grows
Messaging	WhatsApp Business Cloud API; email as fallback	Template messages, delivery webhooks
AI assistant (optional)	LLM with structured output	Parses requests and PDF layouts; never writes timetable data itself
Ops	Docker Compose, structured logs, health checks	Reproducible deployment; observable notification pipeline

6. Data model (built and tested)

The schema is in **schema.sql** and was loaded into PostgreSQL 16 with the seed data. Main groups of tables:

Group	Tables
Reference	faculty, faculty_aliases, rooms, room_aliases, subjects, periods, lab_blocks, student_groups
Master timetable	timetable_versions (with upload hash), timetable_sessions (with as-printed room and faculty), session_groups, session_faculty
Date-specific operations	disruptions, room_policy_overrides, rescheduling_runs, schedule_overrides
Delivery and governance	notification_jobs, audit_log
Views and functions	v_session_detail, v_room_conflicts, v_faculty_conflicts, v_group_conflicts, groups_overlap(), effective_timetable(date), effective_sessions(), effective_conflicts(), blast_radius(), classroom_capacity_check()

Design rules

- **Master is immutable once ACTIVE.** Triggers reject any edit or delete (tested); a corrected upload becomes a new version, and only one version can be ACTIVE (partial unique index). Raw room and faculty text is stored as printed.
- **Sessions, not cells.** Lectures attach to a division; practicals to a batch; electives to a cohort; joint classes link one session to several groups.
- **Student overlap is computed, not assumed.** groups_overlap() walks the group tree: different divisions never overlap; ancestor and descendant always overlap; branches below a common ancestor are disjoint only if they share the same partition key.
- **Intervals are [start, end)**, so a session ending at 10:00 does not clash with one starting at 10:00.

- **Unknown stays unknown.** Missing room or faculty is NULL with needs_review set; the system never invents data.
- **Approval is enforced by the database:** an override cannot be APPROVED or PUBLISHED without an approver.
- **Idempotent notifications:** notification_jobs.dedupe_key is unique, so a retry can never double-send.

7. Rescheduling engine

The solver proposes; the validator checks; a human approves. The AI never sits in this path.

Hard constraints (must hold)

- A room hosts at most one session at a time (temporary rooms only inside their available window; closed rooms never used).
- A faculty member teaches at most one session at a time and is not marked unavailable.
- No student group is in two sessions at once, using the group-overlap rule above.
- Suitability: lab sessions stay in labs (capacity and equipment rules can be added per room).
- Multi-period blocks (2-hour labs) move as a unit; teaching periods only (no recess or lunch); fixed sessions never move.
- Frozen: any session not affected by a disruption keeps exactly its master time, room, faculty and groups.

Soft objectives (minimised, weighted)

- Number of changed sessions (heaviest weight).
- Prefer a room change at the same time over a time change, so students' schedules stay stable.
- If time must change: same day, nearest free period; prefer the same lab type.
- Avoid creating gaps for students and spreading changes over many faculty.

Algorithm

- **0. Freeze:** every session is locked to the master unless a disruption hits it.
- **1. Detect:** affected sessions = master sessions on that weekday whose room or faculty is hit by a disruption window.
- **2. Build one whole-day model** (all divisions and years). Candidates per session: its master slot, the same time in another suitable room (including temporary extra rooms), or another free period the same day; for a hit session also cancel and flag for a human.
- **3. Solve** with CP-SAT: one option per session, room, faculty and student-group occupancy across all divisions, minimise the weighted cost (unaffected sessions are very expensive to move). Return the best few plans. Section 8 explains how ripples are handled.
- **4. Explain infeasibility:** if no plan exists (e.g. every suitable lab is closed), say which constraint blocks it instead of failing silently.
- **5. Validate independently:** re-check the complete effective timetable for the date with plain SQL (effective_conflicts), not solver code, before approval is even enabled. Only NEW clashes block; the master's own clashes show as pre-existing.

Room types and lab blocks

- **Separate lists in the input:** classrooms and labs are given as two lists (Appendix A) and stored in one rooms table with a type.
- **Labs run in fixed 2-hour blocks:** 09:00-11:00, 11:15-13:15 and 14:00-16:00 (plus one Friday 15:00-17:00 session in the master). All 123 lab sessions in the master are exactly 2 hours, so a lab closure removes whole blocks.
- **Classrooms are booked by 1-hour period.** A practical that moves to a classroom keeps its 2-hour block.

- **Default room types:** practical = lab, lecture = classroom, tutorial = classroom or KRC (the master already seats three tutorial batches in three classrooms).

Moving practicals into classrooms (any year)

- **Works for every year.** A room rule in the input (stored as room_policy_overrides) names a year (SY, TY or Final Year), a division or a batch, and a date, and allows classrooms for practicals. Without a rule, practicals only use labs.
- **Same slot, same faculty:** only the room changes, which is the smallest departure from the master.
- **One classroom per batch by default.** Batches run in parallel with their own faculty (the master already does this for tutorials). Merging batches into one classroom changes the master's structure and needs a named human confirmation, enforced by the database.
- **Capacity pre-check:** classroom_capacity_check(date, year) compares rooms needed with free classrooms per block. A shortfall is filled from extra classrooms in the input, then by moving a batch to another free block; anything left is reported as unresolved.
- **Equipment:** practicals that depend on lab machines are flagged and need human confirmation before moving to a classroom.

Year	Lab blocks per week	Blocks that fit	Most rooms needed at once	Free classrooms	Worst shortfall
SY	10	1	9	0 to 5	7
TY	8	1	9	0 to 5	7
Final Year	3	0	6	1 to 2	4

If every lab batch of a year moved to a classroom in its current block, using only classrooms with no printed session. TY and Final Year elective lectures have no printed room, so real availability may be lower. Extra classrooms from the input raise the free count.

The master's own 22 room and 19 faculty clashes (Section 3) are the accepted baseline: ReSlot never edits or hides them. They stay visible as pre-existing; any change to them is a new, approved master version.

8. Handling ripple effects across divisions and years

A change in one division rarely stays there: rooms, faculty and students are shared across the whole department. ReSlot therefore never solves one division in isolation.

Shared thing	Example from the master	How it is handled
Rooms and labs	Thu 09:00-11:00: Research Lab A-212 is used by SY-A SA1, Final A A2 and Final B B2	One occupancy constraint per room across all divisions
Faculty	NK teaches in SY, TY and Final Year	One occupancy constraint per faculty member across all years
Student groups	Lab batches and elective cohorts of one division can share students	Overlap rule (groups_overlap): a student is never in two places
Joint classes	PE-I BACKEND (SSP) is one class for TY-A, B and C	One session with many groups; it moves for all of them together

Approach: one whole-day model

- **One model per date.** Load every session of that weekday for all 8 divisions (about 55) and solve them together with CP-SAT; at this size it takes seconds.
- **Default to the master.** Each session starts at its master slot. Changing a session hit by the disruption is cheap, changing an unaffected session is very expensive, a fixed session cannot change.
- **Ripple depth emerges.** If a free room or period exists, only the hit session moves (zero ripple). Another division's session is displaced only when nothing else works, and the solver picks the displacement that disturbs the fewest people.

- **Diff, not guesswork.** The plan is compared with the master for that date; the difference is the blast radius (blast_radius() in the schema).
- **Messages follow the diff.** Everyone in the diff is notified, in any division or year, not only the division where the problem was reported.

Worked example (checked on the loaded master)

Thursday 22 Oct 2026, 09:00-11:00: A-212 closes. Three batches in two years are hit at once: SY-A SA1 (SZ), Final A A2 (SW) and Final B B2 (SS). Only three other labs are free in that period (A-207, A1-113, B-108), so the batches compete for them and must be placed together. In a test plan using exactly those three labs, blast_radius() listed the three changes with their divisions and faculty, and effective_conflicts() reported no new clash; the 12 clash entries the master already has on that Thursday stayed visible as pre-existing. This was a hand-made test plan, not solver output, and lab suitability (equipment) still has to be checked.

Safeguards

- If a plan moves more sessions than a set limit, or touches more than a set number of divisions, the HOD must confirm it.
- The coordinator can pin a session (never move it) or forbid a room, then re-solve.
- Several disruptions for one date are solved together; if entered one after another, each approved plan becomes the baseline for the next.
- If no plan exists, the system explains which constraint blocks it; it never publishes a partial fix.

9. Notification design

Messages are generated from the approved overrides, so they always match the published timetable.

Recipient	Content	Rule
Each affected faculty member	Every change that affects them on that date: subject, group, old and new time and room, reason	One message per person per publish run (aggregated), not one per session
Each affected student group	Their changed sessions for the date, with faculty name	Sent to the batch or division contact list that has opted in
HOD / coordinator	Summary of all changes and delivery status	Sent after publish

Illustrative faculty message (not a solver output):

```
Timetable update for Prof. R.G. Khedkar - Mon 19 Oct 2026. Your OS lab for SY-B / SB2
(09:00-11:00) has moved from ML Lab (A-210) to PG Lab (A1-208). Same time. Reason: A-210
unavailable. Contact the timetable coordinator with queries.
```

Reliability and compliance

- Outbox pattern: jobs are written in the same transaction as the publish, then sent by a worker with retries and exponential back-off.
- Delivery webhooks update status (sent, delivered, read, failed); failures fall back to email.
- WhatsApp business-initiated messages need pre-approved templates and recipient opt-in; store consent per contact. Check the current WhatsApp Business Platform rules and pricing before building, and develop against a sandbox number.
- Phone numbers are personal data: restrict access by role and store only what is needed.

10. AI assistant (after the deterministic core works)

An administrator types: *"ML lab is unavailable on Monday from 9 to 1."* The assistant extracts a structured request and asks for confirmation when something is ambiguous (which date? which lab?):

```
{"kind": "ROOM_UNAVAILABLE", "room": "A-210", "date": "2026-10-19", "from": "09:00", "to":
"13:15"}
```

- The backend validates the JSON, resolves aliases (ML, MLL, AML to A-210) and shows a confirmation card.
- Only after confirmation is a disruption created; the solver, validator and approval flow then run as normal.
- The assistant has **no write path** to timetable tables. It can propose; it cannot publish.

11. Roles, security and enterprise features

Role	Can do
Timetable coordinator	Upload and verify master, enter disruptions, run the solver, edit proposals
HOD	Approve and publish, view audit log
Faculty	View own timetable and changes, receive notifications
Student / class representative	View division timetable, receive notifications

- Authentication with role-based access control; every change written to audit_log with actor and detail.
- Versioned master data, stored originals for every override, approval records.
- Idempotent, observable notification pipeline (metrics for queue depth, failures, delivery rate).
- Extension hooks (department identifier on core tables) so other departments can be added later without redesign - but not built into the MVP.

12. Roadmap

Milestone	Deliverable	Done when
M0 Data foundation (done)	schema.sql, seed.sql, conflict views, effective_timetable()	Loads cleanly; SQL conflict views agree with an independent check
M1 Ingestion and review	Upload PDF / Excel / CSV on every run; snapshot by file hash; review screen showing each flagged row beside its source cell; saved decisions re-applied	The 62 flagged rows are resolved once; re-uploading the same file needs no review
M2 Availability manager	Classroom and lab lists, lab blocks; mark rooms, labs and faculty unavailable by date and time; add temporary rooms and labs; room rules (practicals in classrooms)	Disruptions persist and show on a calendar
M3 Conflict detection	Clash list for any date with source references	Lists the master's known clashes as pre-existing and flags only new clashes introduced by an override
M4 Rescheduler	CP-SAT whole-day engine, ranked plans, blast-radius report, move cap, infeasibility explanation, independent validator	All test scenarios in Section 13 pass
M5 Approve, publish, notify	Approval queue, per-division/faculty/room timetable views, outbox worker, WhatsApp integration, delivery webhooks	End-to-end demo on sandbox numbers
M6 Extras	Natural-language entry, calendar (.ics) feeds, room-utilisation analytics	Agent cannot bypass solver or approval

Rough estimate: 10 to 14 weeks of part-time work for M1 to M5 by a small team. This is a planning estimate, not a commitment.

13. Testing and evaluation

Scenario	Expected result
One lab closed for a morning (e.g. A-210 on a Monday; the master has 3 sessions there)	Every affected session rehomed or flagged; no new clash; fewest moves
Faculty unavailable for a day	Their sessions flagged for cancellation or manual handling; others untouched
Extra lab added for a date	Solver prefers it over changing times
Several rooms closed in the same period	Minimal time shifts; plan or clear explanation
All suitable labs closed	Infeasible, with the blocking constraint explained; nothing published
Same disruption submitted twice	No duplicate overrides and no duplicate messages
Move that would clash with another year's lab use	Rejected by solver and by validator
A-212 closed Thursday 09:00-11:00 (three batches, three divisions, two years)	All three placed together; blast radius lists every division and faculty member; no new clash
A disruption in one division forces a change in another year	Change shows in the blast radius; the other year's faculty and groups are notified
Two disruptions for one date, entered together vs one after another	Both end in a valid plan; the second run starts from the first approved plan
Same master uploaded again	Existing snapshot reused; saved decisions applied; no new version
All practicals of a year (SY, TY or Final Year) moved to classrooms	Capacity pre-check shows the shortfall per block; extra classrooms or other blocks used; unresolved batches reported
Merge all batches of a division into one classroom	Refused until a named person confirms

Targets: 100% of published timetables pass the validator; plan generated in seconds for a single date; disruption entry to messages queued in minutes (excluding approval time); delivery rate and failure reasons reported.

14. Current status and next steps

Built and tested: schema.sql (PostgreSQL 16), seed.sql (312 sessions, 67 groups, 17 rooms, 34 faculty codes), master_timetable.txt (reviewable source of the seed), build_seed.py (parser, seed generator and independent clash checker), conflicts_report.md (22 room and 19 faculty clashes with source references). The SQL conflict views returned the same counts as the Python checker, and effective_timetable() was exercised with a sample published override. The master lock (all 6 attempted edits rejected) and the new-clash validator (3 override cases: free room, busy room, clashing time) were also tested, together with blast_radius() on a three-division closure, the lab-block and room-rule constraints, and the classroom capacity pre-check for SY, TY and Final Year.

Not built yet: any user interface, the solver, WhatsApp integration, the AI assistant.

- Review the 62 flagged rows and the conflicts report against the PDF (fix master_timetable.txt, re-run build_seed.py).
- Confirm the master's own clashes are the accepted baseline (ReSlot never edits them; a correction needs a new, approved master version).
- Start M1: Spring Boot project on this schema, import endpoint, review screen.

15. Risks and mitigations

Risk	Mitigation
Master mis-read, leading to wrong clashes or wrong messages	Human verification, source references, versioning, golden tests against the known clash list
Master has genuine clashes of its own	Treat as the accepted baseline, show as pre-existing, block only new clashes; change only via a new approved version
WhatsApp approval, policy or cost delays	Submit templates early, sandbox first, email fallback
Solver changes too much	Minimal-change objective, cap on moves, mandatory human approval
A change ripples into many divisions	Whole-day model, blast-radius report, move cap with HOD confirmation, pin and forbid controls
Not enough classrooms when practicals move	Capacity pre-check, extra classrooms from the input, other free blocks, explicit unresolved list
Availability changes between proposal and publish	Re-validate at approval and again at publish time
Personal data (phone numbers)	Consent per contact, role-based access, minimal storage
Scope creep (more departments, more features)	Single-department MVP; extension hooks only
AI misreads a request	Confirmation card, structured validation, no write path to the timetable

Appendix A: Input format (excerpt)

Classrooms and labs are separate lists. Rules and closures are date-specific. A rule's scope can be a year (`{"level": "SY"}`, TY or BE), a division, or a single batch. The full example is in input_example.json.

```
{
  "date": "2026-10-22",
  "classrooms": [{"id": "A-216"}, {"id": "A1-202"}, ...],
  "labs": [{"id": "A-210", "name": "ML / AI-ML Lab", "aliases": ["ML", "AML"]}, ...],
  "lab_blocks": ["09:00-11:00", "11:15-13:15", "14:00-16:00"],
  "unavailable": [{"room": "A-212", "from": "09:00", "to": "11:00"}],
  "extra_rooms": {"classrooms": [{"id": "X-1", "from": "09:00", "to": "17:00"}], "labs": []},
  "faculty_unavailable": [{"faculty": "SW", "from": "09:00", "to": "17:00"}],
  "room_rules": [{"scope": {"level": "SY"}, "sessions": "LAB",
                  "allow": ["CLASSROOM"], "layout": "ONE_ROOM_PER_BATCH"}]
}
```